Lecture Topics

- Introduction to Computing
- Regular Expressions
- Finite Automata
- Equivalence of Models
- Automata Minimisation
- Pumping Lemma, Closure Properties, Algorithms for Regular Languages
- Context-free Grammars
- Chomsky Normal Forms → Concatness Proofs?
- Pushdown Automata
- Pumping lemma, closure properties, deterministic pushdown automata
- Syntax Analysis, CYK algorithm
- Turing machines
- Undecidability, Reductions
- Further undecidability results (PCP), Rice's theorem
- Universal Turing machine, Semidecidability
- P vs NP
- Cook's theorem
- Further NP-complete problems
- Beyond NP

Lecture 1

- Word: Finite string of Symbols (One Java Program)
- Language: Set of words (Set of all correct Java Programs)
- Alphabet: Finite set of symbols allowed in Word Σ
  - Σ^* all possible words over an Alphabet = Language

Operations on strings:

- Concatenation: $u \cdot v$ → $abc \cdot cda = abccda$
- Repetition: u^n → $(abc)^2 = abc \cdot abc = abcabc$
  - → $(abc)^0 = \epsilon$

Inductive Definition of Strings:

- We have set Σ^* → all possible strings over Σ
  - Σ^* can be defined inductively:
    - → $\epsilon \in \Sigma^*$
    - → if $w \in \Sigma^*$ and $\sigma \in \Sigma$ (Symbol) then $w\sigma \in \Sigma^*$ (Inductive Step)

Example

Inductively define the function $\#_a$, which assigns to each string w from $\{a, b\}^*$ the amount of appearances of the character a in w:

- $\#_a(\epsilon) \stackrel{\text{def}}{=} 0$
- $\#_a(w\sigma) \stackrel{\text{def}}{=} \begin{cases} \#_a(w) + 1 & \text{if } \sigma = a \\ \#_a(w) & \text{if } \sigma = b \end{cases}$

Substrings:

Example

The string *abab* has the

- Prefixes ϵ, *a*, *ab*, *aba*, *abab*
- Suffixes ϵ, *b*, *ab*, *bab*, *abab*
- Substrings ϵ, *a*, *b*, *ab*, *ba*, *aba*, *bab*, *abab*

Interval Notation:

Example

Let w = *acbbcabba*, then

- $w[3] = b$
- $w[4, 6] = bca$
- $w[6, 6] = a$
- $w[7, 6] = \epsilon$

Regular Expressions:

→ able to express

→ 3 Properties:

- The **repetition** is expressed through a superscript $*$:
  - $b^* \equiv$ "arbitrarily many *b*s"
  - This corresponds to the language $\{\epsilon, b, bb, bbb, bbbb, \dots\}$
- The **concatenation** is written like a product:
  - $ab^* \equiv$ "one *a* followed by an arbitrary number of *b*s"
  - This corresponds to the language $\{a, ab, abb, abbb, \dots\}$
- The **choice** is expressed through a $+$:
  - $b + c^* \equiv$ "one *b* or an arbitrary number of *c*s"
  - This corresponds to the language $\{b, \epsilon, c, cc, ccc, \dots\}$

↳ Inductive Definition of Regular Expressions:

For each RE α over Σ:

$L(\alpha)$ is defined:

$L(\emptyset) = \emptyset$ ← Empty Set

$L(\epsilon) = \{\epsilon\}$ ← Empty Word

$L(\sigma) = \sigma$ for all $\sigma \in \Sigma$

if α, b are RE's:

$L(\alpha\beta) = L(\alpha) \circ L(\beta)$

$L(\alpha + \beta) = L(\alpha) \cup L(\beta)$

$L((\alpha^*)) = L(\alpha)^*$

$L_1 \circ L_2 = \{uv \mid u \in L_1, v \in L_2\}$

$L_1^* = \{u_1 \dots u_n \mid u_1, \dots, u_n \in L\}$

↳ Kleene Closure

→ "Punkt vor Strich":

() > * > • > +

Definition (Extended Regular Expression)

Abbreviation	Stands for	Explanation
$[a\text{–}z]$	$a + \dots + z$	one of $a, \dots, z$, a and z need to be alphabet symbols
$\alpha?$	$(\alpha + \epsilon)$	α once or never
α^+	$\alpha\alpha^*$	α at least once
α^n	$\alpha \dots \alpha$	α repeated n times
$\alpha^{(m,n)}$	$\alpha^m(\alpha?)^{n-m}$	α at least m times and at most n times

→ Equivalence of RE's:

- → $\alpha \equiv \beta$ iff $L(\alpha) = L(\beta)$ → They decide the same Language
- → Proove $\alpha \equiv \beta$ with a counter-example
- → Proove $\alpha \equiv \beta$:

1. Associativity of $\{+, \cdot\}$:
   - ↳ $\alpha + (\beta + \gamma) \equiv (\alpha + \beta) + \gamma$
   - ↳ $\alpha(\beta\gamma) \equiv (\alpha\beta)\gamma$
2. Commutativity of $\{+\}$:
   - ↳ $\alpha + \beta \equiv \beta + \alpha$
3. Neutral Elmts of $\{+, \cdot\}$:
   - $\alpha + \emptyset \equiv \alpha \equiv \emptyset + \alpha$
   - $\alpha\epsilon \equiv \alpha \equiv \epsilon\alpha$
4. Distributivity
   - $\alpha(\beta + \gamma) \equiv \alpha\beta + \alpha\gamma$
5. Idempotence of $\{*\}$
   - $(\alpha^*)^* \equiv \alpha^*$

Lecture 2

Finite Automata:

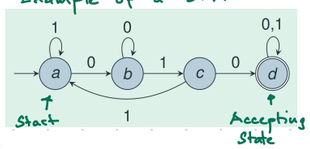
↳ System with finite amount of states. Input causes a change of State.

↳ FA decides a Language

↳ Deterministic if:

1. Single Transition for every Symbol
2. No empty-string Transitions
3. Single starting state

Example of a DFA:



Definition (Deterministic Finite Automaton, DFA)

- A **finite automaton** $\mathcal{A}$ is a quintuple consisting of
  - a finite set Q of **states**,
  - an **input alphabet** Σ,
  - a **transition function** $\delta : Q \times \Sigma \to Q$,
  - an **initial state** $s \in Q$, $|s| = 1$
  - a set $F \subseteq Q$ of **accepting states**
- We write $\mathcal{A} = (Q, \Sigma, \delta, s, F)$

→ Sink State = Non-Accepting state that can't be escaped

→ From RE's to DFA's

→ Some strings are impossible to explain in a DFA

↳ f.e. $(0+1)^*(00(00)^* + 001)0$

↳ solved by Non-determinism

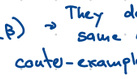
↳ Can be made deterministic later (NFA = Intermediate state)

- A **symbol** σ of a regular expression is translated into a single transition:

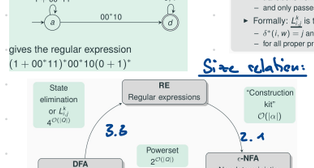
- The **concatenation** of symbols corresponds to the successive operation of transitions, e.g. for 010:

- The **choice** corresponds to a branch, e.g. for 10 + 01:

- The **repetition** corresponds to a loop in the automata, e.g. for $(010)^*$:



Nondeterministic Finite Automata:



a transition relation $\delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$

→ Accepts if a run to (3) exists (one single run is enough)

→ Transition Relation: $\delta \subseteq Q \times \Sigma \times Q$

→ Run Semantics: $a \xrightarrow{1} a \xrightarrow{1} a \xrightarrow{\epsilon} b \dots$

→ NFA' can contain ϵ-transitions

↳ Leads to even more non-determinism (allow clear repetitions)

→ For every RE α $\exists$ ϵ-NFA $\mathcal{A}$ such that $L(\alpha) = L(\mathcal{A})$

↳ Construct over "divided-and-conquer"

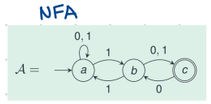
From NFA to DFA:

→ Theorem 3.1

→ For every NFA $\mathcal{A}$ $\exists$ a DFA $\mathcal{A}_D$ with $L(\mathcal{A}_D) = L(\mathcal{A})$

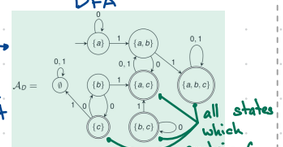
→ DFA will have more states

NFA



construct a state for every Subset of states

DFA

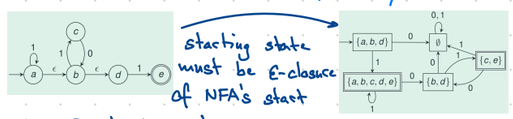


all states which contain C are accepting

From ϵ-NFA to DFA:

→ Theorem 3.2

→ Also here... a DFA exists for every $\mathcal{E}$-NFA



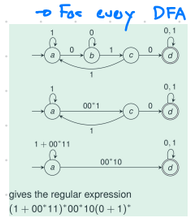
starting state must be $\mathcal{E}$-closure of NFA's start

From DFA's to RE's:

→ Theorem 3.3

→ For every DFA $\mathcal{A}$ $\exists$ RE α such that $L(\alpha) = L(\mathcal{A})$

→ state elimination to transfer



gives the regular expression $(1 + 00^*11)^*00^*10(0 + 1)^*$

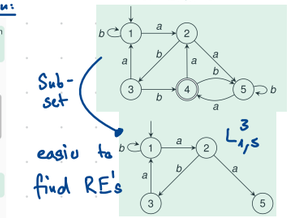
Proof

- Let $\mathcal{A} = (Q, \Sigma, \delta, s, F)$
- Without loss of generality, let $Q = \{1, \dots, n\}$ and $s = 1$
- For each $i, j \in \{1, \dots, n\}$ and $k \in \{0, \dots, n\}$ we want L_{ij}^k to be the set of all strings $w \in \Sigma^*$ for which the automaton
  - transitions from state i to state j,
  - and only passes states from $\{1, \dots, k\}$ in between
- Formally: L_{ij}^k is the set of all strings w with:
  - $\delta^*(i, w) = j$ and
  - for all proper prefixes $v \neq \epsilon$ of w: $\delta^*(i, v) \leq k$

Set of word which makes Automaton transition i→j through at max k

Size relation:

Sub-set



easier to find RE's

Lecture 3/4

Automata Minimization:

→ There is a smallest equivalent DFA for every DFA

→ Minimization: (1) Remove unreachable states
(2) Merge States with Transitions lead to same state
(4) Sink States can be merged
(5) Merge States with same accepting behaviour

→ Equivalence Relation: $xz \in L \iff yz \in L$ → $x \sim_L y$ Nerode Relation

↳ By reading x or y, we get to a state from which continuing every word will lead to the same state

↳ states can be merged

↳ Equivalence classes of $\sim_L$ define states of minimal DFA for L ⇒ equivalence class Automaton

Definition (Equivalence Relation)

- A binary relation R over a set A is called
  - **reflexive** ⇔ for all $x \in A$: $(x, x) \in R$
  - **symmetric** ⇔ for all $x, y \in A$: $(x, y) \in R \Rightarrow (y, x) \in R$
  - **transitive** ⇔ for all $x, y, z \in A$: $(x, y) \in R, (y, z) \in R \Rightarrow (x, z) \in R$
- A binary relation that is reflexive, symmetric and transitive is an **equivalence relation**

Example

- The "same semester" relation:
  - A: The set of all students
  - $(x, y) \in R$, if x and y are in the same semester
- The "congruence modulo d" relation:
  - A: The set $\mathbb{N}$ of all natural numbers
  - $(x, y) \in R$, if $x \equiv_d y$
- The notation $x \equiv_d y$ means that x and y have both the same remainder when divided by d
- Example: $14 \equiv_6 8$

Im in same Sem. as myself

Im in same sem as you → you also with me

Im with you, you with your colegue → im with your colegue

→ Equivalence Class: Class of Elmt which are equivalent

→ A DFA $\mathcal{A}$ with k states has #equivalen classes $\leq k$

Theorem 4.3 (Myhill-Nerode)

A language L is regular if and only if $\sim_L$ has a finite number of equivalence classes

↳ Example of proving Non-Regularity:

Example

- We compute the equivalence classes of $L_{ab^n} = \{a^i b^n \mid n \geq 0\}$
- E.g. it holds: $a^i b^n \sim_{L_{ab^n}} a^j b^n \sim_{L_{ab^n}} a^j b^j \cdots \sim_{L_{ab^n}}$ has the classes:
  - $B_n = \{a^{i+n} b^i \mid i \geq 1\}$, for every $k \in \mathbb{N}_0$,
  - $A_n = \{a^n\}$, for every $k \in \mathbb{N}_0$,
  - $C = \{a^i b^j \mid i < j\} \cup \overline{L(a^* b^*)}$, the class of all strings for which no extension exists that is in L_{ab^n}
- Notation:
  - Let L be a language over Σ and $v \in \Sigma^*$
  - $L/v = \{z \in \Sigma^* \mid vz \in L\}$

Example (cont.)

- To verify that these are the equivalence classes of L_{ab^n}, we need to show:
  - (1) Every string appears in a class $\checkmark$
  - (2) For all strings u, v in the same class it holds that $u \sim_{L_{ab^n}} v$
  - (3) For the strings u, v from different classes it holds that $u \not\sim_{L_{ab^n}} v$
- Instead of (2) and (3), we can equivalently show:
  - (2') All strings v of a class have the same set L_{ab^n}/v
  - (3') Different classes have different sets L_{ab^n}/v
- For every k:
  - For $v \in B_k$, $L_{ab^n}/v = \{b^k\}$
  - For $v \in A_k$, $L_{ab^n}/v = \{a^i b^{k-i} \mid i \geq 0\}$
- For $v \in C$, $L_{ab^n}/v = \emptyset$
- ⇒ The equivalence classes are correctly stated
- ⇒ infinitely many classes
- ⇒ L_{ab^n} is not regular

Proposition 4.5

If $\mathcal{A}$ is a DFA for a language L and has as many states as $\mathcal{A}_L$, then it holds that $\mathcal{A} \cong \mathcal{A}_L$

Isomorphia

Definition (Isomorphism of DFAs, $\cong$)

- Let $\mathcal{A}_1 = (Q, \Sigma, \delta_1, s_1, F_1)$ and $\mathcal{A}_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$ be DFAs over the same input alphabet
- $\mathcal{A}_1$ and $\mathcal{A}_2$ are called **isomorphic** if a bijection $\pi : Q_1 \to Q_2$ exists where:
  1. $\pi(s_1) = s_2$,
  2. for all $q \in Q_1$ it holds that: $q \in F_1 \iff \pi(q) \in F_2$, and
  3. for all $q \in Q_1$ and $\sigma \in \Sigma$ it holds that: $\pi(\delta_1(q, \sigma)) = \delta_2(\pi(q), \sigma)$
- Notation: $\mathcal{A}_1 \cong \mathcal{A}_2$

→ Isomorphism: $\mathcal{A}_1$ and $\mathcal{A}_2$ differ only in Name of states. → π is a state renaming func

Algorithm to get Minimal Automaton: (Table Filling Algo)

→ Recap: Equivalence Class Automaton $\mathcal{A}_L$ is a minimal DFA for L

- How can $\mathcal{A}_L$ be constructed from $\mathcal{A}$?
- The proof of Proposition 4.2 gives us a hint
- If $\mathcal{A}$ is a DFA for L, then $\delta^*(s, x) = \delta^*(s, y)$ implies $x \sim_L y$

⇒ $\mathcal{A}_L$ can be constructed by merging states from $\mathcal{A}$

- Two states p, q from $\mathcal{A}$ can be merged, if they are equivalent according to:
  - for all $z \in \Sigma^*$: $\delta^*(p, z) \in F \iff \delta^*(q, z) \in F$

⇒ To construct the minimal DFA we only need to calculate which states can be merged

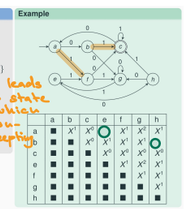
- The basic idea behind our algorithm is to first calculate which states can *not* be merged
- Therefore, we consider an algorithm that calculates the set $N(\mathcal{A})$ of **non-equivalent states**:
  $N(\mathcal{A}) = \{(p, q) \mid p, q \in Q, \exists z : (\delta^*(p, z) \in F \Leftrightarrow \delta^*(q, z) \in F)\}$
- The calculation is done inductively:
  - Begin by computing pairs which are inequivalent due to $z = \epsilon$.
  - Afterwards, compute pairs which are inequivalent due to transitions to other inequivalent pairs

Table-Filling Algorithm

- Input: $\mathcal{A} = (Q, \Sigma, \delta, s, F)$
- Output: Relation $N(\mathcal{A})$

1. $N :=$ $\{(p, q), (q, p) \mid p \in F, q \notin F\}$
2. $N' :=$ Same trans leads to state which is Non-accepting
   $\{(p, q) \notin N \mid \exists \sigma \in \Sigma : \{\delta(p, \sigma), \delta(q, \sigma)\} \in N\}$
3. $N := N \cup N'$
4. If $N' \neq \emptyset$, continue with 2
5. Output N

Example



	a	b	c	e	f	g	h
b	■		X^0	■	X^1	X^1	X^1
c	■	■		X^0	X^1	X^1	X^1
e	■	■	■		X^1	X^1	X^1
f	■	■	■	■		X^1	X^1
g	■	■	■	■	■		X^1
h	■	■	■	■	■	■	

→ Empty State represents pairs which are Equivalent

→ X^k → Nonequiv on k'th iteration

→ Runtime = $O(|Q|^2 \cdot |\Sigma|)$

Minimization Algorithm for DFA

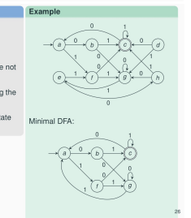
- Input: $\mathcal{A} = (Q, \Sigma, \delta, s, F)$
- Output: minimal DFA $\mathcal{A}$ with $L(\mathcal{A}') = L(\mathcal{A})$

1. Remove all states of $\mathcal{A}$ which are not reachable from s
2. Compute the relation $N(\mathcal{A})$ using the table-filling algorithm
3. Merge iteratively all unmarked state pairs into one state

- Runtime complexity for the minimization algorithm:
  1. $\mathcal{O}(|\delta|) \in \mathcal{O}(|Q|^2|\Sigma|)$
  2. $\mathcal{O}(|Q|^2|\Sigma|)$ using a smart implementation
  3. $\mathcal{O}(|Q|^2|\Sigma|)$

  In total: $\mathcal{O}(|Q|^2|\Sigma|)$

Example



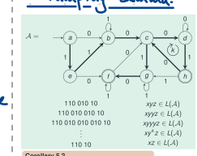
Entire Journey from RE to minimal DFA:

1. Specify the language by a regular expression α
2. Convert α into an ϵ-NFA $\mathcal{A}_1$
3. Convert $\mathcal{A}_1$ into a DFA $\mathcal{A}_2$
4. Convert $\mathcal{A}_2$ into a minimal DFA $\mathcal{A}_3$

Lecture 5

Pumping Lemma:

New method that (sometimes) can proove that a Language is not regular



→ Basic Idea: A cycle can be repeated arbitrary many times

→ If $\mathcal{A}$ is regular: There exists a natural number n such that for each w with $|w| \geq n$ there exists a decomposition $w = xyz$ with
(1) $y \neq \epsilon$ and
(2) $|xy| \leq n$
there exists a $k \geq 0$ such that $xy^kz \notin L$.
Then L is not regular

Corollary 5.2

- Let L be a language
- Assume that for each $n > 0$ there exists a string $w \in L$ with $|w| \geq n$ such that for each decomposition $w = xyz$ with
  (1) $y \neq \epsilon$ and
  (2) $|xy| \leq n$
  there exists a $k \geq 0$ such that $xy^kz \notin L$.
- Then L is not regular

→ The compartimisation so that for each w with $|w| \geq n$ of x, y, z can not w can be looked at as xyz: be choosen? 1,2 must hold $y \neq \epsilon$, $|xy| \leq n$, $xy^kz \in L$

→ Intuition: a Language is not regular if reading a word requires some sort of "counting"

Theorem 5.3 (Jaffe, 78)

A language L is regular **if and only if** there exists an n such that each string $w \in L$ with $|w| \geq n$ can be written as $w = xyz$ in at least one way, such that the following conditions hold:

(1) $y \neq \epsilon$

(2) $|xy| \leq n$

(3') for all $k \geq 0$: $xy^kz \sim_L xyz$

Closure of regular Languages:

↳ These Properties let us "mess" with automatons to construct new ones...

↳ Synthesis of Automata

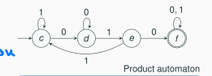
Theorem 5.4

- Let $\mathcal{A}_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ and $\mathcal{A}_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$ be DFAs
- Then, DFAs can be constructed for the following languages:
  (a) for $L(\mathcal{A}_1) \cap L(\mathcal{A}_2)$ with $|Q_1| \cdot |Q_2|$ states in time $\mathcal{O}(|Q_1| \cdot |Q_2| \cdot |\Sigma|)$
  (b) for $L(\mathcal{A}_1) \cup L(\mathcal{A}_2)$ with $|Q_1| \cdot |Q_2|$ states in time $\mathcal{O}(|Q_1| \cdot |Q_2| \cdot |\Sigma|)$
  (c) for $\Sigma^* - L(\mathcal{A}_1)$ with $|Q_1|$ states in time $\mathcal{O}(|Q_1|)$

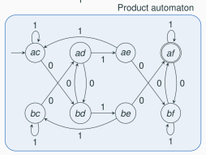
Corollary 5.5

The class of regular languages is closed under intersection, union and complementation.

Product Automaton



Product automaton



→ Synthesis can also be concatenation and iteration: If $\mathcal{A}_1$ and $\mathcal{A}_2$ are DFA's (or NFA's) for $L_1 \& L_2$: There are Automatons for $L_1 \circ L_2$ and L_1^*

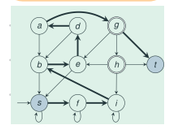
Size (|Q|) of resulting Autom.:

- Q_1 and Q_2 denote the set of states of the automata for L_1 and L_2, respectively

	DFA → DFA	DFA → NFA	NFA → NFA
$L_1 \cap L_2$	$\mathcal{O}(\lvert Q_1\rvert \times \lvert Q_2\rvert)$	$\mathcal{O}(\lvert Q_1\rvert \times \lvert Q_2\rvert)$	$\mathcal{O}(\lvert Q_1\rvert \times \lvert Q_2\rvert)$
$L_1 \cup L_2$	$\mathcal{O}(\lvert Q_1\rvert \times \lvert Q_2\rvert)$	$\mathcal{O}(\lvert Q_1\rvert \times \lvert Q_2\rvert)$	$\mathcal{O}(\lvert Q_1\rvert + \lvert Q_2\rvert)$
$L_1 - L_2$	$\mathcal{O}(\lvert Q_1\rvert \times \lvert Q_2\rvert)$	$\mathcal{O}(\lvert Q_1\rvert \times \lvert Q_2\rvert)$	$\lvert Q_1\rvert \times 2^{\mathcal{O}(\lvert Q_2\rvert)}$
$L_1 \circ L_2$	$\mathcal{O}(\lvert Q_1\rvert \times \lvert Q_2\rvert)$	$\mathcal{O}(\lvert Q_1\rvert \times \lvert Q_2\rvert)$	$\mathcal{O}(\lvert Q_1\rvert + \lvert Q_2\rvert)$
L_1^*	$2^{\mathcal{O}(\lvert Q_1\rvert)}$	$\mathcal{O}(\lvert Q_1\rvert)$	$\mathcal{O}(\lvert Q_1\rvert)$

Further Algorithms for Automatons:

→ Emptyness Test: Is $L(\mathcal{A}) \neq \emptyset$?



→ Forget all edge labels

→ All accepting states lead to one further state (t)

→ Try to find a "way" to (t)

→ Word Problem: Is $w \in L$?

- **RE**: Transform the RE into a ϵ-NFA and then simulate the powerset automaton

⇒ Complexity: $\mathcal{O}(|w| \times |\alpha|)$

- The algorithm for the word problem and thus also the complexity depend on the representation of the language
- **DFA**: Simulate the DFA

⇒ Complexity: $\mathcal{O}(|w| + |\delta|)$

- **NFA**: Simulate the powerset automaton, without constructing it explicitly
  - While doing so, only save the currently reached set of states

⇒ Complexity: $\mathcal{O}(|w| \times |\delta|)$

→ Equivalence Tests: Is $L(\mathcal{A}_1) = L(\mathcal{A}_2)$?

Definition (Equivalence Problem for DFAs)

Input: DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$
Question: Is $L(\mathcal{A}_1) = L(\mathcal{A}_2)$?

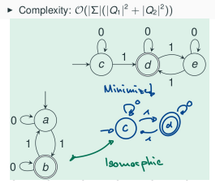
We look at two methods to solve this:

(1) With minimal automata

(2) With the product automaton

(1) With minimal automata:

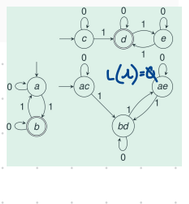
- Construct the minimal automata: $\mathcal{A}_1'$ and $\mathcal{A}_2'$ of $\mathcal{A}_1$ and $\mathcal{A}_2$
- Test if $\mathcal{A}_1'$ and $\mathcal{A}_2'$ are isomorphic:
  - Construct step-wise a bijection π from (the states of) $\mathcal{A}_1'$ to $\mathcal{A}_2'$
  - Initialization: π maps the initial state onto the initial state
  - Then: Continue π according to the transitions of $\mathcal{A}_1'$ and $\mathcal{A}_2'$
- Complexity: $\mathcal{O}(|\Sigma|(|Q_1|^2 + |Q_2|^2))$



(2) With the product automaton:

- Construct the product automaton $\mathcal{A} = \mathcal{A}_1 \times \mathcal{A}_2$ with the accepting set $F = \{(p_1, p_2) \mid (p_1 \in F_1, p_2 \notin F_2)$ or $(p_1 \notin F_1, p_2 \in F_2)\}$
- ⇒ $\delta_{\mathcal{A}}^*((s_1, s_2), w) \in F$ if and only if w is accepted by exactly one of the DFAs
- Thus: $L(\mathcal{A}_1) = L(\mathcal{A}_2)$ if and only if $L(\mathcal{A}) = \emptyset$
- The question "Is $L(\mathcal{A}_1) = L(\mathcal{A}_2)$?" can then be answered by an emptiness test for $\mathcal{A}$

⇒ Complexity: $\mathcal{O}(|Q_1| \times |Q_2| \times |\Sigma|)$



→ Finiteness Test:

Is $|L(\mathcal{A})| = \infty$

↳ Pumping Lemma

Theorem 5.7

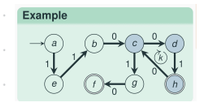
The language of a DFA $\mathcal{A}$ is infinite if and only if $\mathcal{A}$ contains a state q with the following properties:

(a) q is reachable from s,
i.e.: $\exists x \in \Sigma^* : \delta^*(s, x) = q$

(b) q lies on a cycle,
i.e.: $\exists y \in \Sigma^* : y \neq \epsilon$ and $\delta^*(q, y) = q$

(c) An accepting state is reachable from q,
i.e.: $\exists z \in \Sigma^* : \delta^*(q, z) \in F$

Example



↳ $k \in \mathbb{N}$

Lecture 6

→ **Context Free Grammar (CFG)**: Describe a Language with "Memory"

↳ CFG is a rule set to describe a Context Free Language

Definition (Context-free Grammar, CFG)

A **context-free grammar** (**CFG**) $G = (V, \Sigma, S, P)$ consists of

- a finite set V of **variables**
- an alphabet Σ disjoint from V
- a **start variable** $S \in V$
- a finite relation P of **rules**: $P \subseteq V \times (V \cup \Sigma)^*$

Example $L_{Palindrome}$:

$G = (\{S\}, \{a, b\}, S, P =$ { $S \to \epsilon$, $S \to a$, $S \to b$, $S \to aSa$, $S \to bSb$ }) (Rule)

→ A Derivation Step is one application of a rule leading to one Derivation in the end: $u_0 \Rightarrow_G u_1 \Rightarrow_G \dots \Rightarrow_G u_n$ (only terminals)

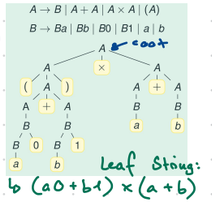
→ The Complete Language CFL is defined as: $L(G) = \{w \in \Sigma^* \mid S \Rightarrow_G^* w\}$ (Derivation Step)

→ Common Standards: ① All Symbols on left-hand Side = Symbols ② First Symbol is the Starting Symbol

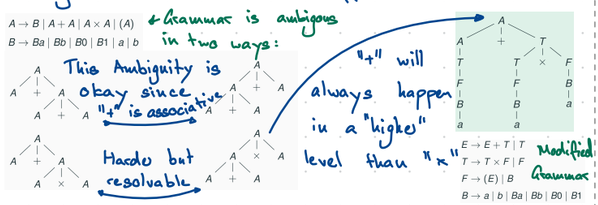
→ **Derivation Trees**: Can Visualize Derivations

$S \Rightarrow_G^* w$ ⇔ There exists a DT with root S an Leaf string w

→ DT can be built in Left-most or Right-most manner. They result in the same leaf string.



→ **Ambiguity**: A CFG is ambigous if it allows Strings which admitt two different DT's



→ Intuition Ambiguity: $L(A) = \{a^i b^j c^k \mid i = j \text{ or } j = k\}$ ... with $i = j = k$ two criteria are fullfilled, so two derivation trees

→ Ambiguity can normally be proovem through induction

→ **Chomsky Hierarchy**: CFG's are subclass of Chomskey Languages

↳ additionally allow strings of terminals and non-terminals on the left hand side of a rule $aBC \to Db$

→ **Extended Context Free Grammars**: CFG's with regular expression on the right hand side of a rule

Definition (Extended Context-free Grammar)

An **extended context-free grammar** $G = (V, \Sigma, S, P)$ consists of

- a set V of **variables**
- an alphabet Σ
- a **start symbol** $S \in V$
- a set P, which contains exactly one **rule** $X \to \alpha_X$ for each variable $X \in V$, where α_X is a regular expression over $V \cup \Sigma$

→ $\alpha_1 | \dots | \alpha_k$ corresponds to $\alpha_1 + \dots + \alpha_k$

Proof idea:

- "(a) ⇒ (b)": ✓
- "(b) ⇒ (a)":
  - Let G' be an extended context-free grammar for L with start symbol S
  - Let $X \to \alpha_X$ be a rule from G'
  - ⇒ α_X is a regular expression
  - ⇒ $L(\alpha_X)$ is a regular language over $V \cup \Sigma$
  - ⇒ $L(\alpha_X)$ is context-free (as seen in a later chapter)
  - For each X let G_X be a grammar for $L(\alpha_X)$ with start symbol X
  - Let G be the grammar which results from the union of the grammars G_X with start symbol S
  - Claim: $L(G) = L(G')$

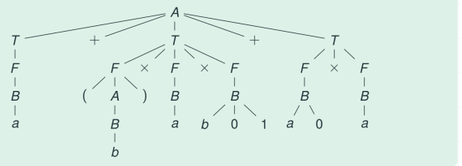
Example: Extended Grammar

$A \to T[+T]^*$
$T \to F[\times F]^*$
$F \to (A) \mid B$
$B \to [a \mid b][a \mid b \mid 0 \mid 1]^*$

⟺

$A \to A + T \mid T$
$T \to T \times F \mid F$
$F \to (A) \mid B$
$B \to a \mid b \mid Ba \mid Bb \mid B0 \mid B1$

Example: Derivation Tree



Lecture 7

Normal Forms: A way to express Grammars in a unified way by restricting the Production Rules

Chomsky Normal Form (CNF):

→ Theorem 7.1: every Grammar describing a CFL can be brought into CNF

Definition (Chomsky Normal Form, CNF)

$G = (V, \Sigma, S, P)$ is in **Chomsky normal form** if

- G only contains useful symbols
- All rules of G are of the form
  - $X \to YZ$ or
  - $X \to \sigma$

  with $X, Y, Z \in V, \sigma \in \Sigma$
- If S does not appear on the right-hand side of any rule, the rule $S \to \epsilon$ is allowed (so that $G \Rightarrow^* \epsilon$ is possible)

↳ Grammars normally grow

(G_1) Only useful symbols are allowed

(G_2) Right-hand sides contain exactly one terminal symbol or only variables

(G_3) No $X \to \beta$ rules with $|\beta| > 2$ are allowed (Procedure)

(G_4) No ϵ-rules are allowed (to transform to CNF)

(G_5) No $X \to Y$ rules are allowed

Five Steps to the Chomsky Normal Form:

① Elimination of Useless Symbols → First remove non-generating and generating

↳ Symbols must be reachable and generating

Example: G_1

$S \to bDD \mid Ca \mid bc$
$A \to B \mid aCC \mid baD$
$B \to cBD \mid \epsilon \mid AC$
$C \to bD \mid \underline{abA}$
$D \to CD \mid \#$

$V = \{S, A, B, C, D\}$

Example: G_2

$S \to W_bDD \mid CW_a \mid W_bW_c$
$A \to B \mid W_aCC \mid W_bW_aD$
$B \to W_cBD \mid \epsilon \mid AC$
$C \to W_bD \mid \underline{W_aW_A}$
$D \to CD \mid W_\#$
$W_a \to a$
$W_b \to b$
$W_c \to c$

$V = \{S, A, B, C, D, W_a, W_b, W_c\}$ (extra)

$S \to AB \mid C$
$A \to b$
$C \to ab$
$D \to c$

$S \to C$
$C \to ab$

② Separating Variables and Terminals

↳ Right-hand side is either: One Terminal, ϵ or multiple Variables

③ Reducing the right-hand Side: At most two variables on the right side of a rule

$A \to B \mid W_aA_1 \mid W_bA_2$
$A_1 \to CC$
$A_2 \to W_aD$

④ Eliminating ϵ-Rules: ~~$E \to \epsilon$~~

↳ Find all variables Y for which $Y \Rightarrow_G^* \epsilon$

Example: G_3

$S \to W_aS_1 \mid CW_a \mid W_bW_c$
$S_1 \to DD$
$A \to B \mid W_aA_1 \mid W_aA_2$
$A_1 \to CC$
$A_2 \to W_aD$
$B \to W_cB_1 \mid AC$
$B_1 \to BD$
$C \to W_aD \mid W_aC_1$
$C_1 \to BA$
$D \to CD \mid W_\#$
$W_a \to a$
$W_b \to b$
$W_c \to c$

$V_\epsilon = \{B, A, C_1\}$

Example: G_4

$S \to W_aS_1 \mid CW_a \mid W_bW_c$
$S_1 \to DD$
$A \to B \mid W_aA_1 \mid W_aA_2$
$A_1 \to CC$
$A_2 \to W_aD$
$B \to W_cB_1 \mid AC \mid C$
$B_1 \to BD \mid D$
$C \to W_aD \mid W_aC_1 \mid W_a$
$C_1 \to BA \mid B$
$D \to CD \mid W_\#$
$W_a \to a$
$W_b \to b$
$W_c \to c$

4. Remove variables that are now not useful
Here: all variables still useful

⑤ Eliminate Unit Productions:

→ Find all occurances whe one Variable maps to one other variable ~~$X \to Y$~~

→ This has to be done recursively and carefully

$V = \{(B, D), (D, E)\}$

$A \to BC$
$B \to D$
$C \to DF$
$D \to E$
$E \to e$

→

$A \to BC$
$B \to e$
$C \to DF$
$D \to e$

⑥ Adding $S \to \epsilon$: If the Language contains ϵ, this must be added

$\epsilon \in L(G)$

$S \to AS \mid b$
$A \to a$

→

$S' \to AS \mid b \mid \epsilon$
$S \to AS \mid b$
$A \to a$

→ **Correctness Proof for CFG**: How do we proove $L(G) = L$?

→ Soundness: Everything creat

→ Completeness: Everything cr

→ Proof $L_{Pali} = L(G_{Pali})$

$L = \{w \in \{a, b\}^* \mid w^R = w\}$

$G = P \to \epsilon \mid a \mid b \mid aPa \mid bPb$

Proof Sketch: Completeness

- To show: $L_{pali} \subseteq L(G_{pali})$
- We use the following non-inductive palindrome definition:
- A string w of length n is a palindrome iff for all $i \in \{1, \dots, |w|\}$: $w[i] = w[n - i + 1]$
- Let $w \in L_{pali}$ and $n = |w|$
- We show $w \in L(G_{pali})$ by induction over n

$n \in \{0, 1\}$:

⇒ $w \in \{\epsilon, a, b\}$ is derivable ✓

$n \geq 2$:

- Case 1: $w[1] = a$
  - ⇒ $w[n] = a$
  - Let $v = w[2, \dots, n-1]$
  - Claim: v is a palindrome:
  - $v[i] = w[i+1]$ → Def. v
  - $= w[n - (i+1) + 1]$ → w palindrome
  - $= v[n - (i+1)]$ → Def. v
  - $= v[(n-2) - i + 1]$
  - Induction: $P \Rightarrow^* v$
  - ⇒ $P \Rightarrow aPa \Rightarrow^* ava = w$
- Case 2: $w[1] = b$ follows analogously

Proof Sketch: Soundness

- To show: $L(G_{pali}) \subseteq L_{pali}$
- Let $w \in L(G_{pali})$
- ⇒ $P \Rightarrow^k w$ for a $k \geq 1$
- We show $w \in L$ by induction over the derivation length k:
- $k = 1$
  - ϵ, a, b are palindromes ✓
- $k > 1$
  - Case 1: $P \Rightarrow aPa \Rightarrow^{k-1} ava = w$, for a $v \in \{a, b\}^*$
    - ⇒ $P \Rightarrow^{k-1} v$
    - ⇒ $w^R = v$ → Induction
    - ⇒ $w^R = av^Ra = ava = w$
  - Case 2: $P \Rightarrow bPb \Rightarrow^{k-1} bvb = w$, for a $v \in \{a, b\}^*$
    - ⇒ Follows analogously

Lecture 8

→ **Pushdown Automata**: Similar to finite Automaton with a Stack

↳ Read Symbols from left to right and decide if accepting

Definition (Pushdown Automaton, PDA)

A **pushdown automaton** (**PDA**) $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, q_0, \tau_0, F)$ consists of

- a finite set Q of states,
- an input alphabet Σ,
- a **stack alphabet** Γ,
- a finite **transition relation** $\delta \subseteq (Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma) \times (Q \times \Gamma^*)$,
- an initial state $q_0 \in Q$,
- an **initial stack symbol** $\tau_0 \in \Gamma$, and
- a set F of accepting states

→ Automaton Behaviour on next step depends on: → current State P → next input symbol σ → topmost stack symbol τ

→ A transition is of Form: (p, σ, τ, q, w) (current state, next input symbol, Topmost stack Symbol, next state, next stack symbol)

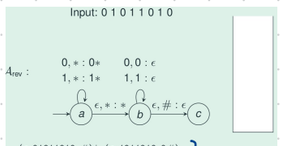
→ PDA's can also be shown as Diagrams:

Notation: $0, * : 0\ast$ (input Symbol σ, Topmost on stack τ, New Stack)

$L_{rev} = \{ww^R \mid w \in \{0,1\}^*\}$

$0, \epsilon : 0$ → erase stack sym

$A_{rev} =$



Definition (Configuration of a PDA)

- Let $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, s, \tau_0, F)$ be a pushdown automaton
- A **configuration** (q, u, v) of $\mathcal{A}$ consists of:
  - a state $q \in Q$
  - the remaining input $u \in \Sigma^*$
  - the stack content $v \in \Gamma^*$
- **Start configuration** with input w: (s, w, τ_0)

→ Successor Configuration, accepting and Run:

Definition (Successor Configuration)

- Let $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, s, \tau_0, F)$ be a PDA
- The successor configuration relation $\vdash_\mathcal{A}$ is defined as follows:
  - For all $p, q \in Q, \sigma \in \Sigma, \tau \in \Gamma, u \in \Sigma^*, z, v \in \Gamma^*$:
  - $(p, \sigma u, \tau v) \vdash_\mathcal{A} (q, u, zv)$ if $(p, \sigma, \tau, q, z) \in \delta$
  - $(p, u, \tau v) \vdash_\mathcal{A} (q, u, zv)$ if $(p, \epsilon, \tau, q, z) \in \delta$
- If $K \vdash_\mathcal{A} K'$ then K' is called a **successor configuration** of K

Definition (Run of a PDA)

- A **run** of a PDA $\mathcal{A}$ is a sequence $K_i, \dots, K_n$ of configurations with $K_i \vdash_\mathcal{A} K_{i+1}$ for all $i \in \{1, \dots, n-1\}$
- Notation: $K_1 \vdash_\mathcal{A}^* K_n$

Note:

- If the input has already been read completely, it is still possible to perform ϵ-transitions
- But: with an empty stack, there is no successor configuration!

Definition (Language of a PDA)

- Let $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, s, \tau_0, F)$ be a pushdown automaton
- $\mathcal{A}$ **accepts a string** $w \in \Sigma^*$, if
- ("accepting with an empty stack") $F = \emptyset$ and $(s, w, \tau_0) \vdash^* (q, \epsilon, \epsilon)$ for a $q \in Q$, or
- ("accepting with accepting states") $F \neq \emptyset$ and $(s, w, \tau_0) \vdash^* (q, \epsilon, u)$ with $q \in F$
- $L(\mathcal{A}) = \{w \mid \mathcal{A} \text{ accepts } w\}$

↳ PDA's can accept via accepting state or empty stack

→ Proof by Simulation:

↳ From Empty Stack to Accepting State:

↳ Add new symbol \$ to stack... as soon as visible this is used for transition $\epsilon, \$: \epsilon$ to AS

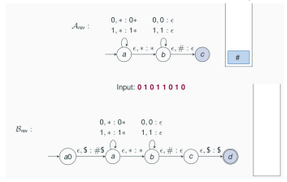
↳ From AS to Empty Stack:

↳ add "clean up - states" to each accepting state

Theorem 8.1

(a) For each pushdown automaton $\mathcal{A}$ that accepts with an empty stack, there exists a pushdown automaton $\mathcal{B}$ that accepts with accepting states such that $L(\mathcal{B}) = L(\mathcal{A})$

(b) For each pushdown automaton $\mathcal{A}$ that accepts with accepting states, there exists a pushdown automaton $\mathcal{B}$ that accepts with an empty stack such that $L(\mathcal{B}) = L(\mathcal{A})$

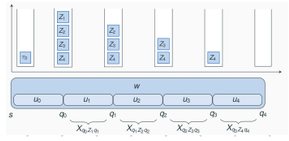


→ PDA's and CFG's: For each CFG, there exists a PDA (and vice versa)

↳ From CFG to PDA is a task of Simulation

↳ To empty the stack again

↳ From PDA to CFG is more complicated:



→ $u_1 .. u_n$ are substrings of w

Lecture 8

→ **Deterministic PDA's**:

Definition (DPDA, Deterministic Context-free)

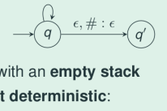
- A pushdown automaton $\mathcal{A} = (Q, \Sigma, \Gamma, \delta, s, \tau_0, F)$ is called **deterministic** (**DPDA**) if for all $p \in Q, \sigma \in \Sigma, \tau \in \Gamma$: $|\delta(p, \sigma, \tau)| + |\delta(p, \epsilon, \tau)| \leq 1$
- A language is called **deterministic context-free** if it is decided by a deterministic pushdown automaton

↳ For each combination (p, σ, τ) there is only one transition (p, σ, τ, q, w) in δ

Example

- As a reminder: the following PDA decides the language $L_{\langle\rangle}$ of the well-formed bracket expressions over $\{\langle a\rangle, \langle/a\rangle, \langle b\rangle, \langle/b\rangle\}$:

$\langle b\rangle, * : \langle b\rangle *$
$\langle a\rangle, * : \langle a\rangle *$
$\langle/b\rangle, \langle b\rangle : \epsilon$
$\langle/a\rangle, \langle a\rangle : \epsilon$

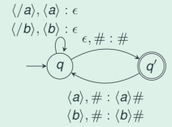


- It decides with an **empty stack**
- But it is **not deterministic**:
  - $(q, \langle b\rangle, \#, q, \langle b\rangle\#) \in \delta$ and
  - $(q, \epsilon, \#, q', \epsilon) \in \delta$
- A critical input string is: $\langle b\rangle\langle a\rangle\langle/a\rangle\langle/b\rangle\langle b\rangle\langle/b\rangle$
- How should it continue after reading $\langle b\rangle\langle a\rangle\langle/a\rangle\langle/b\rangle$?

Example

- A **deterministic** pushdown automaton with **accepting states** for the same language:

$\langle b\rangle, \langle b\rangle : \langle b\rangle\langle b\rangle$
$\langle b\rangle, \langle a\rangle : \langle b\rangle\langle a\rangle$
$\langle a\rangle, \langle b\rangle : \langle a\rangle\langle b\rangle$
$\langle a\rangle, \langle a\rangle : \langle a\rangle\langle a\rangle$
$\langle/a\rangle, \langle a\rangle : \epsilon$
$\langle/b\rangle, \langle b\rangle : \epsilon$



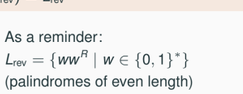
$\langle a\rangle, \# : \langle a\rangle\#$
$\langle b\rangle, \# : \langle b\rangle\#$

- After reading $\langle b\rangle\langle a\rangle\langle/a\rangle\langle/b\rangle$ it transitions into an accepting state
- If the string is not finished yet, it can read further symbols

→ **Correctness Proofs for PDA**:

Example

$\mathcal{A}_{rev}$:



Proposition 8.7

$L(\mathcal{A}_{rev}) = L_{rev}$

- As a reminder: $L_{rev} = \{ww^R \mid w \in \{0,1\}^*\}$ (palindromes of even length)

Proof Sketch

- We prove:
  - $L_{rev} \subseteq L(\mathcal{A}_{rev})$: completeness
  - $L(\mathcal{A}_{rev}) \subseteq L_{rev}$: soundness

Proof Sketch: Completeness

Completeness:

- To show: $L_{rev} \subseteq L(\mathcal{A}_{rev})$
- We show that for arbitrary $w \in \Sigma^*$ the string ww^R is accepted by $\mathcal{A}_{rev}$
- To this end the following can be proved using induction over the length of u and v for arbitrary strings $u, v, x \in \{0,1\}^*$:
  (a) $(a, ux, \#) \vdash^* (a, x, u^R\#)$ and
  (b) $(b, v, v\#) \vdash^* (b, \epsilon, \#)$
- Then it follows:

$(a, ww^R, \#) \vdash^* (a, w^R, w^R\#)$
$\vdash (b, w^R, w^R\#)$
$\vdash^* (b, \epsilon, \#)$
$\vdash (c, \epsilon, \epsilon)$

⇒ $ww^R \in L(\mathcal{A}_{rev})$

Example

$\mathcal{A}_{rev}$:

Proof Sketch (cont.)

Soundness:

- To show: $L(\mathcal{A}_{rev}) \subseteq L_{rev}$
- Clearly: a string v is in L_{rev} if and only if
  - it has an even length $n = 2k$ and
  - for each $i \leq k$: $v[i] = v[n - i + 1]$
- Let $v \in L(\mathcal{A}_{rev})$ with n symbols

⇒ $(a, v, \#) \vdash^* (b, \epsilon, \#) \vdash (c, \epsilon, \epsilon)$

Proof Sketch (cont.)

- From the construction of $\mathcal{A}_{rev}$ this computation proceeds in three phases
  1. $\mathcal{A}_{rev}$ reads a prefix $v[1, k]$ of the input and writes it onto the stack symbol-wise,
  2. then $\mathcal{A}_{rev}$ transitions into the state b,
  3. lastly $\mathcal{A}_{rev}$ reads the remaining input $v[k+1, n]$ and compares it to the symbols previously written onto the stack
- The following can be shown by induction:
- The configuration after phase 1 is $(a, v[k+1, n], v[1, k]^R\#)$
- So that phase 3 is successful the following has to hold:
  - $n - k = k \Rightarrow n = 2k$
  - for each $i \leq k$: $v[i] = v[n - i + 1]$

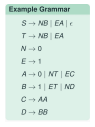
⇒ $v \in L_{rev}$

Example Run backtracking algo:

1 S → aAO
2 S → aB1
3 A → aAO
4 A → aB1
5 A → c
6 B → aAO
7 B → aB1
8 B → c

input: aaac111

S → aB1 2
B → aaB11 7
B → aaaB111 7
B → aaac111 8



CYK: 01110100